Freshmate

Application: Grocery Purchase & Expiry Tracker with Recipe Suggestions
Framework: wxPython (Desktop GUI)

Team Members:

- Ishita Gulati (PES2UG25AM111)
- Janaki Niharika (PES2UG25AM113)
- Hemal S (PES2UG25AM109)
- Harman Chahal (PES2UG25CS200)

Table of Contents

1. Executive Summary
2. Project Overview
3. Key Features
4. Modules & Libraries Used
5. System Architecture
6. Complete Source Code
7. CSV Data Files
8. Usage Workflow
9. Conclusion

Executive Summary

Freshmate is a full-featured desktop application designed to help users manage their grocery inventory efficiently. The application enables users to record grocery purchases with specific dates, automatically track expiration dates based on predefined shelf life data, identify items that are expiring or have already expired, and intelligently suggest recipes using ingredients that are nearing expiration. Built with wxPython, Freshmate combines an intuitive graphical user interface with robust CSV-based data management, providing a practical solution to minimize food waste and optimize grocery usage.

Project Overview

Purpose

Freshmate solves the common problem of food waste by:

- **Recording purchases** with dates for all grocery items
- **Calculating expiry dates** automatically based on shelf life data
- **Alerting users** when items are about to expire
- **Suggesting recipes** that use expiring ingredients
- **Minimizing waste** by helping users consume items before they spoil

Application Architecture

The application follows a clean three-module architecture:

- **main.py**: Entry point that initializes the application
- **storage.py**: Data layer handling CSV I/O and business logic
- **gui.py**: wxPython GUI implementation for user interactions

---

Key Features

1. Purchase Management

- ComboBox dropdown with all available grocery items
- Date picker for recording exact purchase dates
- Automatic expiry date calculation from shelf life
- Real-time display of all purchases with calculated expiry dates

2. Expiry Tracking

- Configurable date range for checking expiring items
- Automatic separation of expiring vs. expired items
- Color-coded lists for easy identification
- Sortable lists by expiry date

3. Recipe Suggestions

- Smart recipe matching algorithm
- Two categories of recipes:
  - **Recipes You Can Make**: All ingredients currently available
  - **Recipes That Need Items**: Missing ingredients listed for purchase
- Automatic filtering to avoid expired ingredients

4. Data Persistence

- All purchase data saved to CSV files
- Reference data (groceries and recipes) easily editable
- User purchases tracked in `user_purchases.csv`
- Data survives application restarts

Modules & Libraries Used

External Libraries

Library	Purpose
wxPython (wx, wx.adv)	Cross-platform GUI framework for windows, panels, buttons, list controls, and date pickers
csv	Reading and writing CSV files with DictReader, reader, and DictWriter
datetime, timedelta	Date arithmetic and formatting for expiry calculations
pathlib.Path	Cross-platform file path handling and existence checks
typing (Dict, List, Tuple)	Type hints for function signatures and IDE support

Table 1: Libraries and Modules

Python Version Requirements

- **Python 3.6 or higher** (for f-strings and type hints support)

System Architecture

Data Flow Diagram

Application Startup:
main.py
├─ Creates wx.App()
├─ Instantiates MainFrame
├─ Loads data (groceries, recipes, purchases)
└─ Runs app.MainLoop() [Event loop]

Purchase Recording:
User Input → on_add_purchase() → save_user_purchase() → CSV Append
↓
load_user_purchases() → refresh_purchases_list() → Display Update

Recipe Matching:
User Input (check date, within_days)
↓
get_expiring_items() [Calculates expiry dates]
↓
find_recipes_using_items() [Matches ingredients]
↓
Populate Four Result Lists

Complete Source Code

main.py – Application Entry Point

Component	Function
main()	Creates wx.App, instantiates MainFrame, shows window, runs MainLoop()

Table 2: main.py Functions

Source Code:

```
import wx
from gui import MainFrame

def main():
app = wx.App(redirect=False)
frame = MainFrame(None)
frame.Show()
app.MainLoop()

if name == "main":
main()
```

---

storage.py – Data Layer

Constants and Configuration:

```
import csv
from datetime import datetime, timedelta
from pathlib import Path
from typing import Dict, List, Tuple

BASE = Path(file).parent
```

CSV File Paths

```
GROCERY_CSV = BASE / "groceries.csv"
RECIPE_CSV = BASE / "recipes.csv"
PURCHASES_CSV = BASE / "user_purchases.csv"

DATE_FMT = "%Y-%m-%d"
```

Functions:

```
def load_groceries() -> Dict[str, int]:
"""Load grocery items and shelf life from CSV."""
d = {}
```

```
if not GROCERY_CSV.exists():
return d
with open(GROCERY_CSV, newline="", encoding="utf-8") as f:
reader = csv.DictReader(f)
for r in reader:
item = r["item"].strip().lower()
try:
d[item] = int(r["shelf_life_days"])
except Exception:
continue
return d

def load_recipes() -> Dict[str, List[str]]:
"""Load recipes and ingredients from CSV."""
recipes = {}
if not RECIPE_CSV.exists():
return recipes
with open(RECIPE_CSV, newline="", encoding="utf-8") as f:
reader = csv.reader(f)
header = next(reader, None)
for row in reader:
if not row:
continue
name = row[0].strip()
ingredients = [c.strip().lower() for c in row[1:7] if c and c.strip()]
recipes[name] = ingredients
return recipes

def save_user_purchase(item: str, purchase_date: str):
"""Append purchase record to CSV file."""
new_file = not PURCHASES_CSV.exists()
with open(PURCHASES_CSV, "a", newline="", encoding="utf-8") as f:
writer = csv.DictWriter(f, fieldnames=["item", "purchase_date"])
if new_file:
writer.writeheader()
writer.writerow({"item": item.lower(), "purchase_date": purchase_date})

def load_user_purchases() -> Dict[str, datetime]:
"""Load user purchases, keeping latest date per item."""
if not PURCHASES_CSV.exists():
return {}
items = {}
with open(PURCHASES_CSV, newline="", encoding="utf-8") as f:
reader = csv.DictReader(f)
for r in reader:
item = r["item"].strip().lower()
try:
dt = datetime.strptime(r["purchase_date"], DATE_FMT)
except Exception:
continue
if item not in items or dt > items[item]:
```

```
items[item] = dt
return items

def get_expiring_items(check_date: datetime,
groceries: Dict[str, int],
purchases: Dict[str, datetime],
within_days: int = 2) -> Tuple[List[Tuple[str, datetime]],
List[Tuple[str, datetime]]]:
"""Calculate expiring and expired items."""
expiring = []
expired = []
for item, p_date in purchases.items():
shelf = groceries.get(item)
if shelf is None:
continue
expiry = p_date + timedelta(days=shelf)
if expiry < check_date:
expired.append((item, expiry))
elif check_date <= expiry <= check_date + timedelta(days=within_days):
expiring.append((item, expiry))
expiring.sort(key=lambda x: x[1])
expired.sort(key=lambda x: x[1])
return expiring, expired

def find_recipes_using_items(expiring_items: List[str],
expired_items: List[str],
user_items: List[str],
recipes: Dict[str, List[str]]) -> Tuple[List[Dict], List[Dict]]:
"""Find recipes matching available and expiring ingredients."""
possible = []
need_buy = []
user_set = set(i.lower() for i in user_items)
exp_set = set(i.lower() for i in expiring_items)
expired_set = set(i.lower() for i in expired_items)
```

```
for rname, ings in recipes.items():
    if expired_set.intersection(ings):
        continue
    if not exp_set.intersection(ings):
        continue
    missing = [i for i in ings if i not in user_set]
    if not missing:
        possible.append({"recipe": rname, "ingredients": ings})
    else:
        need_buy.append({"recipe": rname, "ingredients": ings, "missing": missing}
return possible, need_buy
```

gui.py – User Interface

Import Statements and Setup:

```
import wx
import wx.adv
from datetime import datetime, timedelta
from pathlib import Path
from typing import List
from storage import (
load_groceries, load_recipes, save_user_purchase,
load_user_purchases, get_expiring_items, find_recipes_using_items, DATE_FMT,
)
```

MainFrame Class – Complete Implementation:

```
class MainFrame(wx.Frame):
def init(self, parent=None):
super().init(parent, title="Freshmate", size=(900, 660))
panel = wx.Panel(self)
self.groceries = load_groceries()
self.recipes = load_recipes()
self.user_purchases = load_user_purchases()
```

```
    main_sizer = wx.BoxSizer(wx.VERTICAL)

    # Purchase Entry Panel
    pnl_entry = wx.StaticBoxSizer(
        wx.StaticBox(panel, label="Add Purchase"), wx.VERTICAL)
    grid = wx.FlexGridSizer(2, 4, 8, 8)
    grid.AddGrowableCol(1)

    grid.Add(wx.StaticText(panel, label="Item:"), 0, wx.ALIGN_CENTER_VERTICAI
    choices = sorted(self.groceries.keys())
    self.item_choice = wx.ComboBox(panel, choices=choices, style=wx.CB_DROPI
    grid.Add(self.item_choice, 1, wx.EXPAND)

    grid.Add(wx.StaticText(panel, label="Purchase date:"), 0, wx.ALIGN_CENTER_
    self.date_picker = wx.adv.DatePickerCtrl(panel,
        style=wx.adv.DP_DROPDOWN | wx.adv.DP_SHOWCENTURY)
    grid.Add(self.date_picker, 1, wx.EXPAND)

    add_btn = wx.Button(panel, label="Add & Save")
    add_btn.Bind(wx.EVT_BUTTON, self.on_add_purchase)
```

```
grid.Add(add_btn, 0)

clear_btn = wx.Button(panel, label="Clear Purchases File")
clear_btn.Bind(wx.EVT_BUTTON, self.on_clear_purchases)
grid.Add(clear_btn, 0)

pnl_entry.Add(grid, 0, wx.ALL | wx.EXPAND, 8)
main_sizer.Add(pnl_entry, 0, wx.ALL | wx.EXPAND, 8)

# Purchases List Panel
pnl_list = wx.StaticBoxSizer(
    wx.StaticBox(panel, label="Current Purchases (latest per item)"), wx.VERTI
self.purchases_list = wx.ListCtrl(panel, style=wx.LC_REPORT | wx.BORDER_S
self.purchases_list.InsertColumn(0, "Item", width=200)
self.purchases_list.InsertColumn(1, "Purchase Date", width=140)
self.purchases_list.InsertColumn(2, "Expiry Date", width=140)
pnl_list.Add(self.purchases_list, 1, wx.ALL | wx.EXPAND, 6)
main_sizer.Add(pnl_list, 0, wx.ALL | wx.EXPAND, 8)

# Expiry Check Panel
pnl_check = wx.StaticBoxSizer(
    wx.StaticBox(panel, label="Check Expiry & Recipes"), wx.VERTICAL)
hbox = wx.BoxSizer(wx.HORIZONTAL)
hbox.Add(wx.StaticText(panel, label="Check date:"), 0,
         wx.ALIGN_CENTER_VERTICAL | wx.RIGHT, 6)
self.check_date_picker = wx.adv.DatePickerCtrl(panel,
    style=wx.adv.DP_DROPDOWN | wx.adv.DP_SHOWCENTURY)
hbox.Add(self.check_date_picker, 0, wx.RIGHT, 10)

self.check_within = wx.SpinCtrl(panel, value="2", min=0, max=30)
hbox.Add(wx.StaticText(panel, label="Within days:"), 0,
         wx.ALIGN_CENTER_VERTICAL | wx.LEFT, 8)
hbox.Add(self.check_within, 0, wx.LEFT | wx.RIGHT, 6)

check_btn = wx.Button(panel, label="Check Expiring Items")
check_btn.Bind(wx.EVT_BUTTON, self.on_check_expiry)
hbox.Add(check_btn, 0, wx.LEFT, 8)
```

```
pnl_check.Add(hbox, 0, wx.ALL | wx.EXPAND, 8)

# Results Area
results_sizer = wx.BoxSizer(wx.HORIZONTAL)

v1 = wx.BoxSizer(wx.VERTICAL)
v1.Add(wx.StaticText(panel, label="Expiring Items (soon)"), 0, wx.BOTTOM, 4
self.exp_list = wx.ListCtrl(panel, style=wx.LC_REPORT | wx.BORDER_SUNKEN
self.exp_list.InsertColumn(0, "Item", width=140)
self.exp_list.InsertColumn(1, "Expiry Date", width=120)
v1.Add(self.exp_list, 1, wx.EXPAND | wx.RIGHT, 6)
results_sizer.Add(v1, 1, wx.EXPAND)

v_exp = wx.BoxSizer(wx.VERTICAL)
v_exp.Add(wx.StaticText(panel, label="Expired Items"), 0, wx.BOTTOM, 4)
self.expired_list = wx.ListCtrl(panel, style=wx.LC_REPORT | wx.BORDER_SUN
self.expired_list.InsertColumn(0, "Item", width=140)
self.expired_list.InsertColumn(1, "Expiry Date", width=120)
v_exp.Add(self.expired_list, 1, wx.EXPAND | wx.RIGHT, 6)
results_sizer.Add(v_exp, 1, wx.EXPAND)

v2 = wx.BoxSizer(wx.VERTICAL)
v2.Add(wx.StaticText(panel,
       label="Recipes You Can Make (all ingredients present)"), 0, wx.BOTTOM, 4
self.rec_ok = wx.ListCtrl(panel, style=wx.LC_REPORT | wx.BORDER_SUNKEN
self.rec_ok.InsertColumn(0, "Recipe", width=150)
self.rec_ok.InsertColumn(1, "Ingredients", width=260)
v2.Add(self.rec_ok, 1, wx.EXPAND | wx.RIGHT, 6)
results_sizer.Add(v2, 2, wx.EXPAND)

v3 = wx.BoxSizer(wx.VERTICAL)
v3.Add(wx.StaticText(panel,
       label="Recipes That Need Items (missing listed)"), 0, wx.BOTTOM, 4)
self.rec_need = wx.ListCtrl(panel, style=wx.LC_REPORT | wx.BORDER_SUNKE
self.rec_need.InsertColumn(0, "Recipe", width=150)
self.rec_need.InsertColumn(1, "Missing Items", width=260)
v3.Add(self.rec_need, 1, wx.EXPAND)
results_sizer.Add(v3, 2, wx.EXPAND)
```

```
    pnl_check.Add(results_sizer, 1, wx.ALL | wx.EXPAND, 6)
    main_sizer.Add(pnl_check, 1, wx.ALL | wx.EXPAND, 8)

    self.status = wx.StaticText(panel, label="Ready.")
    main_sizer.Add(self.status, 0, wx.ALL | wx.EXPAND, 8)

    panel.SetSizer(main_sizer)
    self.Bind(wx.EVT_CLOSE, self.on_close)
    self.refresh_purchases_list()

def on_add_purchase(self, event):
    item = (self.item_choice.GetValue() or "").strip().lower()
    if not item:
        wx.MessageBox("Choose an item to add.", "Error", wx.ICON_ERROR)
        return
    d = self.date_picker.GetValue()
    try:
        dt_text = d.FormatISODate()
    except Exception:
        dt_text = datetime.now().strftime(DATE_FMT)
    save_user_purchase(item, dt_text)
    self.user_purchases = load_user_purchases()
    self.refresh_purchases_list()
    self.status.SetLabel(f"Saved: {item} @ {dt_text}")

def on_clear_purchases(self, event):
    try:
        p = Path("user_purchases.csv")
        if p.exists():
            p.unlink()
        self.user_purchases = {}
        self.refresh_purchases_list()
        self.status.SetLabel("Cleared user_purchases.csv")
    except Exception as e:
        wx.MessageBox(f"Failed to clear purchases: {e}", "Error", wx.ICON_ERROR

def refresh_purchases_list(self):
```

```
    self.user_purchases = load_user_purchases()
    self.purchases_list.DeleteAllItems()
    for item, dt in sorted(self.user_purchases.items()):
        shelf = self.groceries.get(item, None)
        expiry_str = "-"
        if shelf is not None:
            expiry_dt = dt + timedelta(days=shelf)
            expiry_str = expiry_dt.strftime(DATE_FMT)
        idx = self.purchases_list.InsertItem(self.purchases_list.GetItemCount(), iter
        self.purchases_list.SetItem(idx, 1, dt.strftime(DATE_FMT))
        self.purchases_list.SetItem(idx, 2, expiry_str)

def on_check_expiry(self, event):
    d = self.check_date_picker.GetValue()
    try:
        check_date = datetime.strptime(d.FormatISODate(), DATE_FMT)
    except Exception:
        check_date = datetime.now()
    within = int(self.check_within.GetValue())
    purchases = load_user_purchases()
    expiring, expired = get_expiring_items(check_date, self.groceries,
                                           purchases, within_days=within)

    self.exp_list.DeleteAllItems()
    exp_items = []
    for item, expiry in expiring:
        idx = self.exp_list.InsertItem(self.exp_list.GetItemCount(), item)
        self.exp_list.SetItem(idx, 1, expiry.strftime(DATE_FMT))
        exp_items.append(item)

    self.expired_list.DeleteAllItems()
    expired_items = []
    for item, expiry in expired:
        idx = self.expired_list.InsertItem(self.expired_list.GetItemCount(), item)
        self.expired_list.SetItem(idx, 1, expiry.strftime(DATE_FMT))
        expired_items.append(item)

    possible, need_buy = find_recipes_using_items(exp_items, expired_items,
```

```
                                        list(purchases.keys()), self.recipes)

        self.rec_ok.DeleteAllItems()
        self.rec_need.DeleteAllItems()

        for r in possible:
            i = self.rec_ok.InsertItem(self.rec_ok.GetItemCount(), r["recipe"])
            self.rec_ok.SetItem(i, 1, ", ".join(r["ingredients"]))

        for r in need_buy:
            i = self.rec_need.InsertItem(self.rec_need.GetItemCount(), r["recipe"])
            self.rec_need.SetItem(i, 1, ", ".join(r["missing"]))

        self.status.SetLabel(
            f"Found {len(exp_items)} expiring item(s), {len(possible)} ready recipes, "
            f"{len(need_buy)} recipes needing purchase, {len(expired_items)} expired.

    def on_close(self, event):
        self.Destroy()
```

if **name** == "**main**":
app = wx.App(redirect=False)
frm = MainFrame(None)
frm.Show()
app.MainLoop()

CSV Data Files

groceries.csv

Complete reference table of 66 grocery items with shelf life in days:

Raw CSV Format:

item,shelf_life_days
milk,5
cheese,14
tomato,7
potato,30
onion,21
apple,20
banana,5
eggs,21
bread,5

spinach,4
butter,30
carrot,25
cucumber,7
coconut,40
maida,120
sugar,365
curd,7
paneer,5
ghee,120
rice,100
wheat_flour,90
rava_suji,120
besan,90
poha,120
vermicelli,180
toor_dal,180
moong_dal,180
rajma,180
chole,180
green_peas,5
coriander,3
mint,3
cabbage,7
cauliflower,5
beans,7
capsicum,7
tamarind,180
garlic,20
ginger,14
green_chilli,7
lemon,10
grapes,5
mango,6
papaya,5
pepper_powder,240
peanuts,10
watermelon,7
orange,14
pomegranate,20
chicken,3
fish,2
mutton,3
almonds,180
cashews,120
raisins,180
jaggery,180
salt,365
tea_powder,365
coffee_powder,180
turmeric_powder,365

red_chilli_powder,240
cumin_seeds,365
mustard_seeds,365
garam_masala,180
oil,365
cream,5
lady_finger,3

recipes.csv

Collection of 37+ Indian recipes with key ingredients:

Complete Raw CSV:

```
recipe,item1,item2,item3,item4,item5,item6
masala_chai,tea_powder,milk,ginger,,,
lemon_rice,lemon,oil,mustard_seeds,green_chilli,curry_leaves,turmeric_powder
upma,rava_suji,onion,green_chilli,carrot,,
dal_tadka,toor_dal,tomato,onion,garlic,oil,cumin_seeds
paneer_butter_masala,cream,butter,paneer,tomato,garam_masala,
palak_paneer,spinach,paneer,onion,oil,garlic,
aloo_gobi,potato,cauliflower,onion,tomato,garam_masala,
veg_korma,carrot,beans,potato,coconut,oil,garam_masala
egg_bhurji,eggs,onion,tomato,green_chilli,oil,
egg_curry,eggs,onion,tomato,oil,garam_masala,
fish_curry,oil,fish,lemon,garlic,red_chilli_powder,
mutton_curry,garam_masala,mutton,onion,tomato,ginger,
chicken_curry,oil,chicken,onion,tomato,garam_masala,pepper_powder
curd_salad,curd,cucumber,carrot,salt,,
panner_bhurji,paneer,tomato,onion,green_chilli,oil,
spinach_soup,spinach,garlic,onion,salt,,
lemonade,lemon,sugar,,,,
mango_shake,mango,milk,sugar,,,
banana_smoothie,banana,milk,sugar,,,
fried_rice,rice,carrot,beans,capsicum,oil,
jeera_rice,rice,cumin_seeds,oil,onion,,
sambar,toor_dal,tomato,carrot,beans,tamarind,
rasam,tomato,garlic,coriander,salt,tamarind,
dal_fry,moong_dal,onion,tomato,garlic,oil,
chole_masala,oil,chole,onion,tomato,garam_masala,
rajma_curry,oil,rajma,onion,tomato,garam_masala,
aloo_paratha,wheat_flour,potato,green_chilli,coriander,,
veg_pulao,rice,carrot,beans,green_peas,garam_masala,
poha,poha,onion,peanuts,lemon,coriander,
bhindi_fry,lady_finger,onion,oil,turmeric_powder,,
peanut_chutney,peanuts,garlic,green_chilli,,,
coconut_chutney,coconut,green_chilli,ginger,,,
tomato_chutney,tomato,garlic,red_chilli_powder,oil,,
omelette,eggs,oil,pepper_powder,salt,,
curd_rice,curd,rice,ginger,pomegranate,,
aloo_masala,potato,onion,green_chilli,pepper_powder,,
```

friuit_salad,apple,banana,grapes,watermelon,papaya,
toast,bread,butter,,,,

Usage Workflow

Step-by-Step Example

1. Launch Application
$ python main.py
Freshmate GUI window opens with empty purchase list.

2. Add a Purchase

- Select "milk" from Item dropdown
- Choose purchase date (today)
- Click "Add & Save"
- Status shows: "Saved: milk @ [date]"
- Milk appears in purchases list with calculated expiry

3. Add More Items

- Select "tea_powder", add with today's date
- Select "ginger", add with today's date
- Select "curd", add 2 days ago

4. Check Expiry

- Set "Check date" to future date
- Set "Within days" to number of days for expiry window
- Click "Check Expiring Items"

5. View Results

- **Expiring Items**: Items nearing expiration visible with dates
- **Recipes You Can Make**: Recipes with all available ingredients
- **Recipes Need Items**: Recipes showing missing ingredients needed
- Status updates with counts

6. Clear History (Optional)

- Click "Clear Purchases File" to reset all user data

Interface Demo Screenshots

Screenshot 1: Item Selection and Purchase Entry

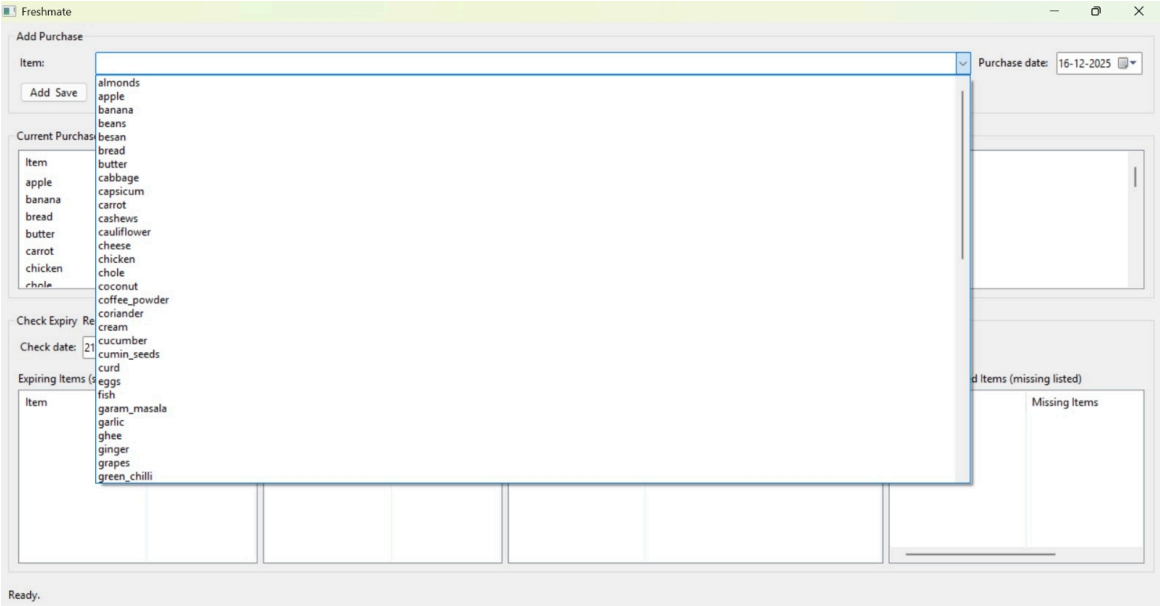

Figure 1: Freshmate Interface - Item Selection and Purchase Entry: Shows the Add Purchase panel with dropdown menu displaying all 66 available grocery items (almonds, apple, banana, beans, besan, bread, butter, cabbage, capsicum, carrot, cashews, coriander, cream, cucumber, cumin seeds, curd, eggs, fish, garlic, ghee, ginger, grapes, green chilli, and more), date picker for recording purchase date, Add & Save button, Clear Purchases File button, and the Current Purchases list showing tracked items with their purchase and calculated expiry dates organized in columnar format.

Key Features Shown:

- ComboBox with alphabetically sorted 66 grocery items
- Date picker pre-populated with current date (16-12-2025)
- Current Purchases table displaying: Item name, Purchase Date, and Expiry Date
- Add & Save and Clear Purchases File action buttons
- Real-time expiry calculation based on shelf life data

Screenshot 2: Recipe Suggestions and Expiry Analysis

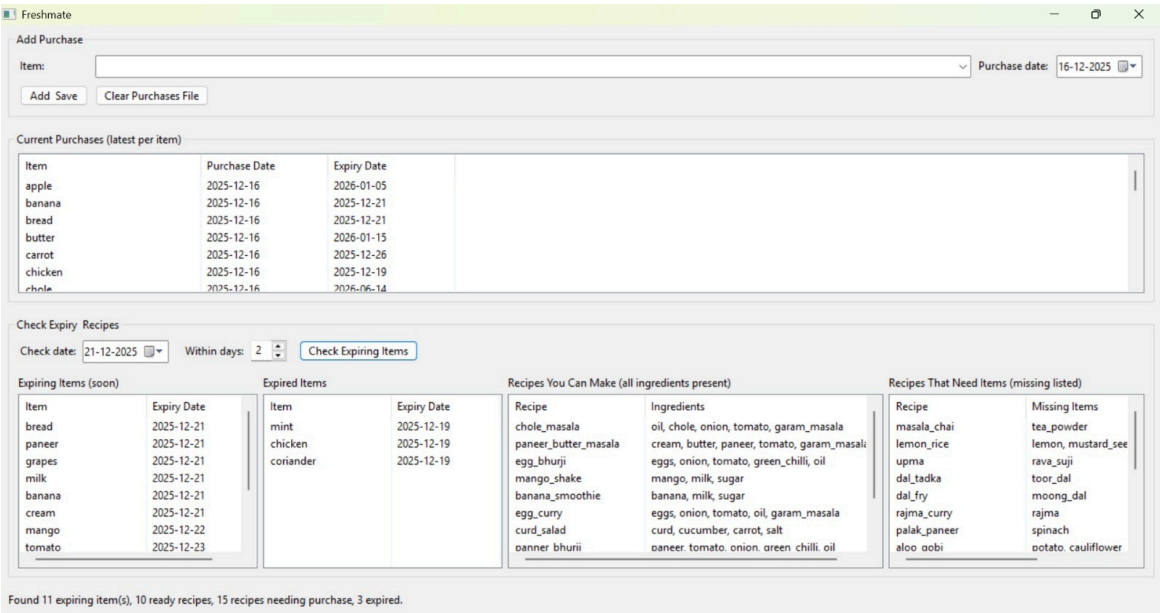

Figure 2: Freshmate Interface - Recipe Suggestions and Expiry Status: Demonstrates the Check Expiry & Recipes panel with date selector (21-12-2025) and within-days parameter (2 days). Shows four categorized result panels: Expiring Items (11 items: bread, paneer, grapes, milk, banana, cream, mango, tomato), Expired Items (3 items: mint, chicken, coriander), Recipes You Can Make with all ingredients present (10 recipes: chole masala, paneer butter masala, egg bhurji, mango shake, banana smoothie, egg curry, curd salad, fried rice, banner bhurii), and Recipes That Need Items showing missing ingredients (masala chai missing tea powder, lemon rice missing lemon and mustard seeds, upma missing rava suji, dal tadka missing toor dal, dal fry missing moong dal, rajma curry missing rajma, palak paneer missing spinach, aloo gobi missing potato and cauliflower).

Key Features Shown:

- Expiry check configuration: Check date (21-12-2025) and Within days window (2)
- Four-panel results layout with real-time categorization
- **Expiring Items (soon)**: 11 items with approaching expiry dates
- **Expired Items**: 3 items that have already passed expiration
- **Recipes You Can Make**: 10 recipes with all ingredients currently available
- **Recipes That Need Items**: Multiple recipes with missing ingredients listed for shopping
- Status bar showing summary: "Found 11 expiring item(s), 10 ready recipes, 15 recipes needing purchase, 3 expired."

Design Principles

1. Separation of Concerns

- **storage.py**: Pure data logic, no GUI dependencies
- **gui.py**: UI layer, calls storage functions
- **main.py**: Minimal setup, entry point only

2. Type Safety

- All functions use type hints (Dict, List, Tuple)
- IDE autocomplete support
- Clear function signatures

3. Data Persistence

- CSV format for easy editing
- Case normalization for consistency
- Latest purchase per item tracking

4. User Experience

- Intuitive dropdown menus
- Real-time calculations
- Clear status messages
- Multi-panel results display

Conclusion

Freshmate demonstrates professional software engineering practices by combining a well-structured data layer (storage.py) with an intuitive graphical interface (gui.py). The three-module architecture ensures maintainability and extensibility, allowing for future enhancements such as database backends, nutritional tracking, or cloud synchronization.

Key Achievements

- ✅ **Clean Architecture**: Separation of concerns across three modules
- ✅ **Smart Recipe Matching**: Intelligent ingredient-based suggestions
- ✅ **Persistent Data**: All purchases saved to CSV files
- ✅ **User-Friendly GUI**: wxPython interface with multi-panel results
- ✅ **Type Safety**: Type hints throughout for code clarity
- ✅ **Real-Time Expiry Tracking**: Automatic date calculations
- ✅ **Minimal Dependencies**: Only wxPython required

Future Enhancement Opportunities

- Database backend (SQLite) for scalability
- Nutritional information tracking
- Recipe ratings and user feedback
- Shopping list generation
- Barcode scanning integration
- Mobile app version
- Cloud sync for household sharing
- Multi-language support

Freshmate successfully addresses food waste by providing an elegant solution for grocery tracking and recipe suggestions based on expiring inventory items.